Introduction to State-Space Representation

Describing a system by first-order state equations $\dot{x}=Ax+Bu$.

Ogata, *Modern Control Engineering*, Ch. 9.

In this tutorial you will:

- write a mass-spring-damper in state-space form and build it with `ss`,
- convert between state-space and transfer function (`tf` / `ss`), and
- compute the free response and meet the state-transition matrix.

Step through with **Ctrl+Enter**, or render a report with `publish`.

A state-space model describes a system by a set of first-order differential equations in the **state vector** $x(t) \in \mathbf{R}^n$, together with an algebraic output equation:

$$\dot{x}(t) = Ax(t) + Bu(t)$$

$$y(t) = Cx(t) + Du(t)$$

where A $(n \times n)$ is the **system matrix**, B $(n \times r)$ the **input matrix**, C $(m \times n)$ the **output matrix**, and D $(m \times r)$ the **direct transmission matrix**. The state $x(t)$ summarizes all information about the past needed (with future inputs) to determine the future response – unlike the transfer function, the state-space form is not restricted to single-input single-output (SISO) zero-initial-condition systems.

Contents

Example: Mass-Spring-Damper in State-Space Form

$m\ddot{y} + b\dot{y} + ky = u$ with $m = 1,\ b = 2,\ k = 5$. Choosing $x_1 = y,\ x_2 = \dot{y}$:

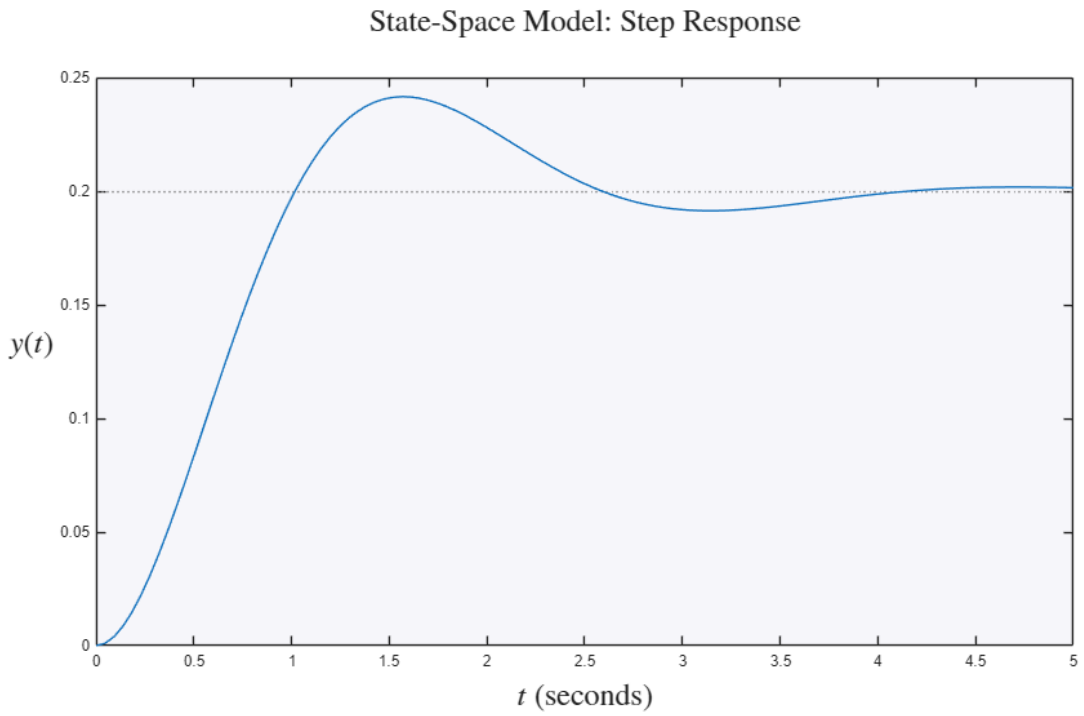
$$\dot{x}_1 = x_2$$

$$\dot{x}_2 = -\frac{k}{m}x_1 - \frac{b}{m}x_2 + \frac{1}{m}u$$

```
m = 1; b = 2; k = 5;
A = [0 1; -k/m -b/m];
B = [0; 1/m];
C = [1 0];
D = 0;
sys_ss = ss(A,B,C,D);
```

The ‘ss‘ object can be queried for its matrices directly, simulated with ‘step‘/‘impulse‘/‘lsim‘, and converted to other model forms.

```
figure
step(sys_ss)
title('State-Space Model: Step Response','Interpreter','latex','FontSize',20)
ylabel('$y(t)$','Interpreter','latex','FontSize',20)
set(get(gca, 'YLabel'), 'Rotation', 0)
xlabel('$t$','Interpreter','latex','FontSize',20)
```



State-Space to Transfer Function

MATLAB computes $G(s) = C(sI - A)^{-1}B + D$ via ‘tf(sys_ss)‘.

```
G_from_ss = tf(sys_ss);
fprintf('Transfer function recovered from state-space:\n')
G_from_ss
```

```
Transfer function recovered from state-space:

G_from_ss =

        1
  -------------
  s^2 + 2 s + 5

Continuous-time transfer function.
```

Direct verification: build the same plant from physical parameters as a transfer function and confirm it matches.

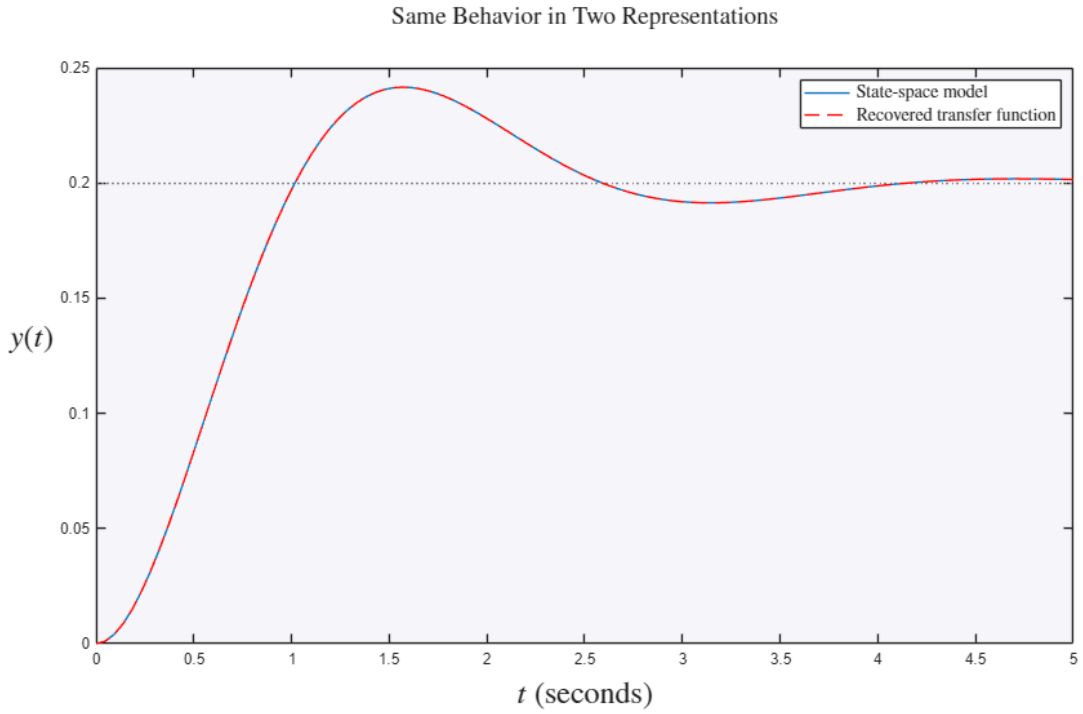
```
G_direct = tf(1,[m b k]);
fprintf('Direct TF poles vs. SS-derived poles:\n')
[pole(G_direct) pole(G_from_ss)]
```

```
Direct TF poles vs. SS-derived poles:

ans =

  -1.0000 + 2.0000i  -1.0000 + 2.0000i
  -1.0000 - 2.0000i  -1.0000 - 2.0000i
```

What Changed vs. What Didn't: Representation vs. Behavior

Converting between state-space and transfer-function form changes the **representation** but not the input-output **behavior**: the step responses of the ss and tf models lie exactly on top of each other.

```
figure
step(sys_ss)
hold on
step(G_from_ss,'r--')
hold off
legend('State-space model','Recovered transfer function','Interpreter','latex','FontSize',12)
title('Same Behavior in Two Representations','Interpreter','latex','FontSize',16)
ylabel('$y(t)$','Interpreter','latex','FontSize',20)
set(get(gca, 'YLabel'), 'Rotation', 0)
xlabel('$t$','Interpreter','latex','FontSize',20)
```



Transfer Function to State-Space

Conversely, 'ss(G)' realizes a transfer function in (typically) controllable canonical form. The realization is **not unique** – infinitely many (A, B, C, D) share the same transfer function, related by similarity transformations $\bar{x} = Tx$ (see `RepresenationTheory.m`).

```
G3 = tf([1 3],[1 4 6 8]);
sys_from_tf = ss(G3);
fprintf('Canonical-form realization of G3:\n')
sys_from_tf.A
sys_from_tf.B
sys_from_tf.C
```

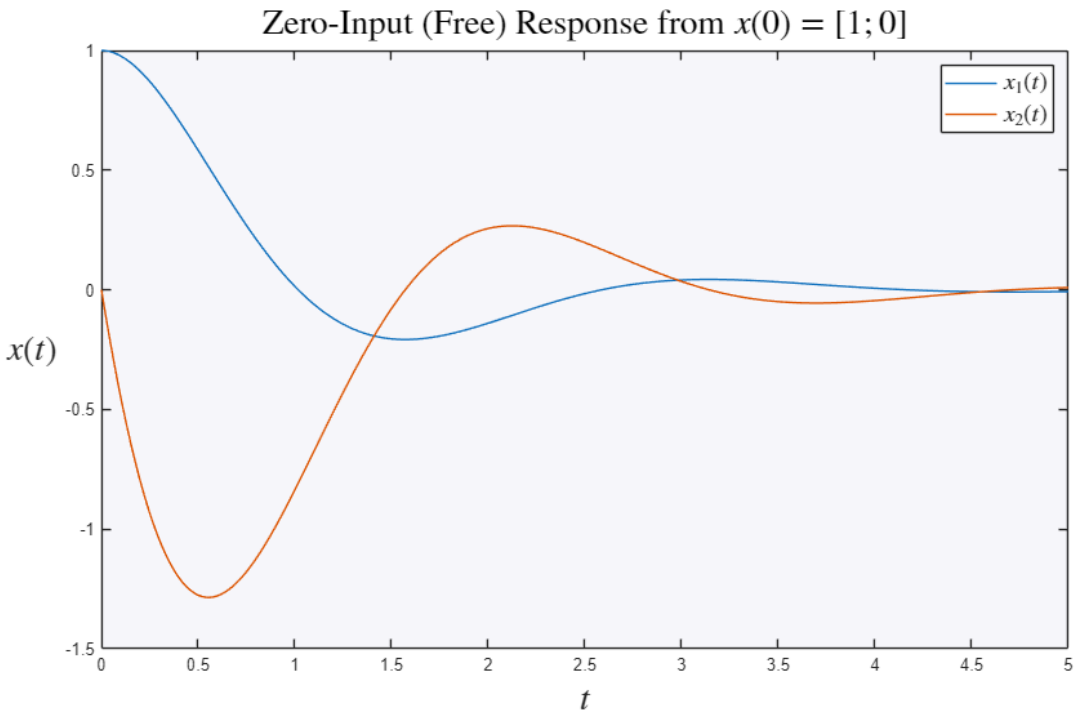
```
Canonical-form realization of G3:

ans =

    -4    -3    -2
     2     0     0
     0     2     0


ans =

     1
     0
     0


ans =

         0    0.5000    0.7500
```

Free (Zero-Input) Response and the State-Transition Matrix

With $u(t) = 0$, $\dot{x} = Ax$ has solution $x(t) = e^{At}x(0)$, where $\Phi(t) = e^{At}$ is the **state-transition matrix**. For a general input, the full solution is the variation-of-parameters formula:

$$x(t) = e^{At}x(0) + \int_0^t e^{A(t-\tau)}Bu(\tau)\,d\tau$$

```
x0 = [1; 0];
t = 0:0.01:5;
[y_free,~,x_free] = initial(sys_ss,x0,t);
figure
plot(t,x_free)
legend('$x_1(t)$','$x_2(t)$','Interpreter','latex','FontSize',14)
title('Zero-Input (Free) Response from $x(0)=[1;0]$','Interpreter','latex','FontSize',20)
ylabel('$x(t)$','Interpreter','latex','FontSize',20)
set(get(gca, 'YLabel'), 'Rotation', 0)
xlabel('$t$','Interpreter','latex','FontSize',20)
```



Try it yourself

- Change the damping `b` to 0.5 and notice the free response ring longer (the state-transition matrix e^{At} decays more slowly).
- Round-trip the model with `tf(ss(G3))` and confirm you recover `G3`.